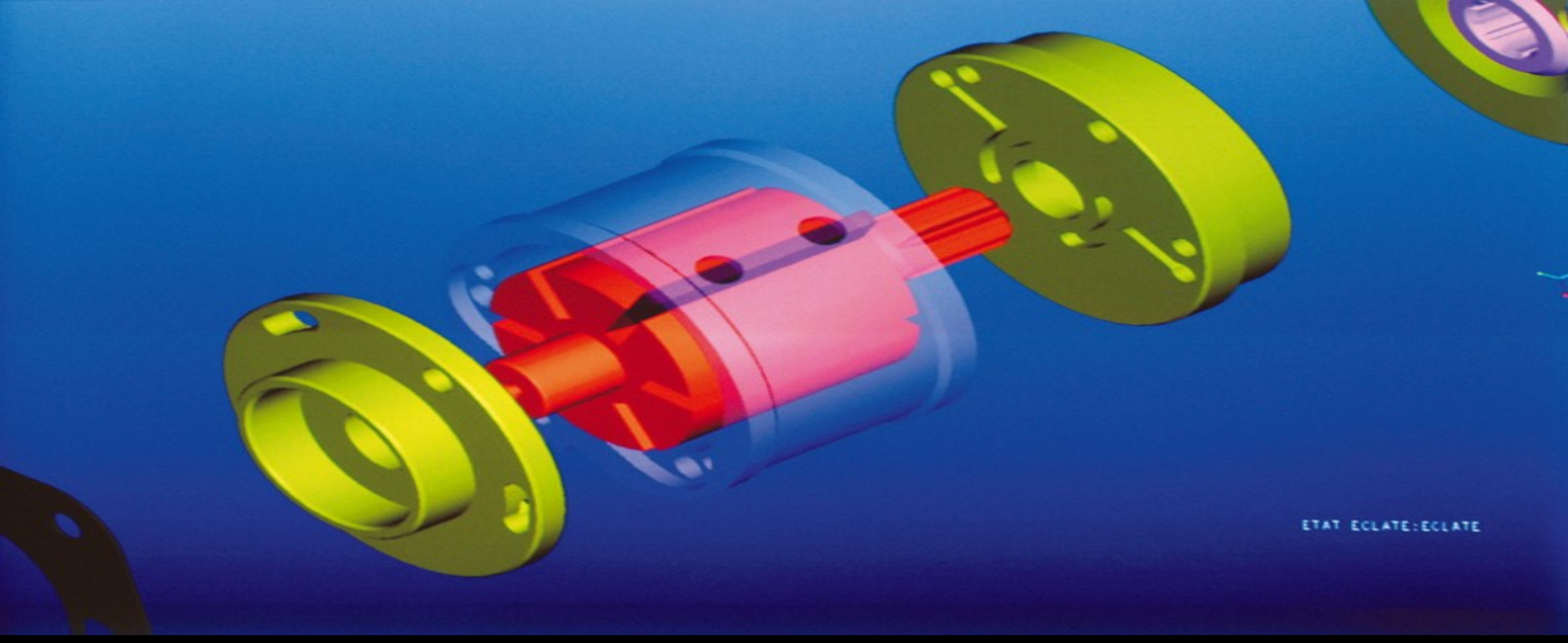


UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Création et optimisation d'un système d'informations de gestion d'un réseau de vélos en libre-service

Rapport de projet BD70 - A26

AUBERT Colin - GRATIA Alban - PREVOST Antoine

Département Génie informatique
3A FISA

utbm université de technologie Belfort-Montbéliard

itii FRANCHE-COMTÉ

UIMM PÔLE FORMATION Franche-Comté LA FABRIQUE DE L'AVENIR

Mots clefs

Base de données - MariaDB - InnoDB - MySQL - Vélo - Optimisation - Automatisation

AUBERT Colin - GRATIA Alban - PREVOST Antoine **Rapport de projet BD70 - A26**

Résumé

Ce rapport présente la conception et la réalisation d'un système d'information pour un réseau de vélos en libre-service : une base de données relationnelle MariaDB en troisième forme normale, dont les règles métier (un vélo à un seul endroit, une borne par vélo, une location à la fois par usager, calcul automatique de la durée et du coût) sont garanties par des contraintes, des déclencheurs et des procédures stockées, et une application web PHP (Slim 4, Twig) qui l'exploite selon trois rôles : usager, opérateur et administrateur. La base a été peuplée avec un jeu de données factices de près de 5 Go (5 millions de locations, 10 millions d'événements historisés). À ce volume, plusieurs statistiques ne répondaient plus en un temps raisonnable : une démarche de mesure, d'analyse des plans d'exécution et de réécriture des requêtes a ramené chaque onglet de la page de statistiques sous 5 secondes sur la période par défaut de 30 jours (de 2,5 à 3,4 s sans cache, moins de 0,1 s avec), avec des gains allant jusqu'à un facteur 70 sur la requête la plus coûteuse. Le rapport détaille ces évolutions et les mesures qui les justifient.

utbm
université de technologie
Belfort-Montbéliard

Table des matières

1 Introduction **5**

1.1 Contexte et objectifs 5

1.2 Organisation et outils 5

1.3 Couverture du cahier des charges 5

2 Analyse des besoins **6**

2.1 Acteurs 6

2.2 Règles métier 7

3 Conception de la base de données **8**

3.1 Modèle conceptuel 8

3.2 Modèle logique 10

3.3 Normalisation 11

3.4 Contraintes d'intégrité 11

3.5 Index 11

4 Implémentation SQL **11**

4.1 Déclencheurs et historisation 11

4.2 Procédures et fonctions métier 12

4.3 Événements planifiés 12

4.4 Sécurité 12

5 Jeu de données factices **13**

6 Application web **13**

6.1 Architecture 13

6.2 Fonctionnalités par rôle 14

6.3 Page de statistiques 14

7 Évolution du projet **14**

7.1 Chronologie 14

7.2 D'un prototype PHP natif à une application structurée 15

7.3 Problèmes rencontrés et solutions 15

8 Analyse des performances **16**

8.1 Protocole de mesure 16

8.2 Constat initial 16
8.3 Configuration du serveur 16
8.4 Réécriture des requêtes 17
8.4.1 Trajets origine-destination 17
8.4.2 Indisponibilité de la flotte 18
8.4.3 Remplissage des stations 19
8.4.4 Index couvrants 19
8.5 Résultats 19
8.5.1 Temps des requêtes principales 19
8.5.2 Montée en charge 20
8.5.3 Temps d'affichage et cache applicatif 21
8.6 Limites et pistes d'amélioration 22

9 Conclusion **22**

A Installation **22**

B Reproduire les mesures **23**

1 Introduction

1.1 Contexte et objectifs

Le projet de l'UV BD70 consiste à concevoir et développer le système d'information d'un réseau de vélos en libre-service à l'échelle d'une agglomération. Le système gère les stations et leurs bornes, la flotte de vélos mécaniques et électriques, les usagers et leurs abonnements, les locations et leur facturation, la maintenance et la régulation du réseau. Il doit fournir à l'exploitant une vision précise de l'utilisation du réseau (trajets, heures de pointe, stations saturées ou vides) afin d'optimiser le rééquilibrage des vélos.

Le sujet insiste sur deux points, qui ont guidé nos choix :

- la **cohérence des données** : un vélo ne peut être qu'à un seul endroit à la fois, une borne n'accueille qu'un vélo, un usager n'emprunte qu'un vélo à la fois ;
- l'**automatisation des calculs** dans la base : durée et coût des locations, historisation des états et des positions, facturation et expiration des abonnements.

1.2 Organisation et outils

- **SGBD** : MariaDB 10.4 (distribution XAMPP), moteur InnoDB ; scripts SQL versionnés (`schema.sql`, `triggers.sql`, `procedures.sql`, `seed.sql`) et migrations ré-exécutables.
- **Application** : PHP 8 avec le micro-framework Slim 4, gabarits Twig, Tailwind CSS et Chart.js pour les graphiques.
- **Gestion de projet** : dépôt Git commun, une branche de développement (`dev`), une branche de documentation (`docs`) et des branches de fonctionnalités.

À rédiger : répartition du travail entre les membres de l'équipe et méthode de travail (réunions, outils de suivi).

1.3 Couverture du cahier des charges

Le tableau 1 met en regard chaque exigence du sujet et sa réalisation ; les parties suivantes en détaillent la conception.

TABLE 1 – Exigences du sujet et réalisation

Exigence	Réalisation	Interface
Inscription, connexion, rôles	tables `utilisateur`, `usager`, `operateur`, `administrateur` ; procédures `sp_inscrire_usager`, `sp_authentifier` ; contrôle d'accès par rôle	oui
Stations et bornes	tables `station`, `borne` ; vue `v_stations_occupation` (vélos disponibles par type, bornes libres, en temps réel) ; création, modification, suppression, recherche et filtres	oui
Flotte	table `velo` (type, mise en service, état, batterie, borne) ; contraintes de position et de batterie ; historiques d'état et de position alimentés par déclencheurs	oui

Exigence	Réalisation	Interface
Abonnements et tarification	tables `formule`, `tranche_tarifaire`, `abonnement`, `facture` ; fonction `fn_cout_formule` ; souscription, renouvellement, résiliation ; expiration par événement planifié	oui
Locations	procédures `sp_emprunter_velo`, `sp_rendre_velo` ; durée calculée par la base, coût calculé par déclencheur ; unicité « une location en cours » par vélo et par abonnement	oui
Signalements et maintenance	tables `signalement`, `intervention` ; file de traitement des signalements ; procédures d'intervention	partielle
Régulation	procédures `sp_stations_a_reequilibrer`, `sp_proposer_deplacements` ; tables `vehicule`, `tournee`, `ligne_tournee`	partielle
Statistiques et rapports	six procédures `sp_stat_*` ; page à onglets, graphiques, export CSV	oui
Ajout, modification, suppression, recherche, filtres	stations, comptes, profil ; filtres sur la flotte et les locations ; pagination	oui
3NF	voir partie 3 (dénormalisations contrôlées justifiées)	–
Données factices en quantité	procédures `sp_seed_*` ; environ 5 Go (partie 5)	–
Sécurité	compte applicatif à droits minimaux (jusqu'à la colonne), mots de passe bcrypt, jetons CSRF, requêtes préparées	–
Rapport de performances	partie 8	–

La maintenance et la régulation ne sont pas exigées dans l'interface par le sujet : elles sont entièrement réalisées dans la base (tables, procédures, données factices), et l'interface en expose une partie (file des signalements, propositions de rééquilibrage).

2 Analyse des besoins

2.1 Acteurs

— **Visiteur** : consulte la disponibilité du réseau, s'inscrit, se connecte.
— **Usager** : souscrit et gère son abonnement, emprunte et restitue des vélos, consulte ses trajets et ses factures, signale un problème.
— **Opérateur** (régulation ou maintenance) : suit la flotte et les locations, traite les signalements, consulte les statistiques et les propositions de rééquilibrage.
— **Administrateur** : droits de l'opérateur, plus la gestion des stations, des bornes et des comptes.

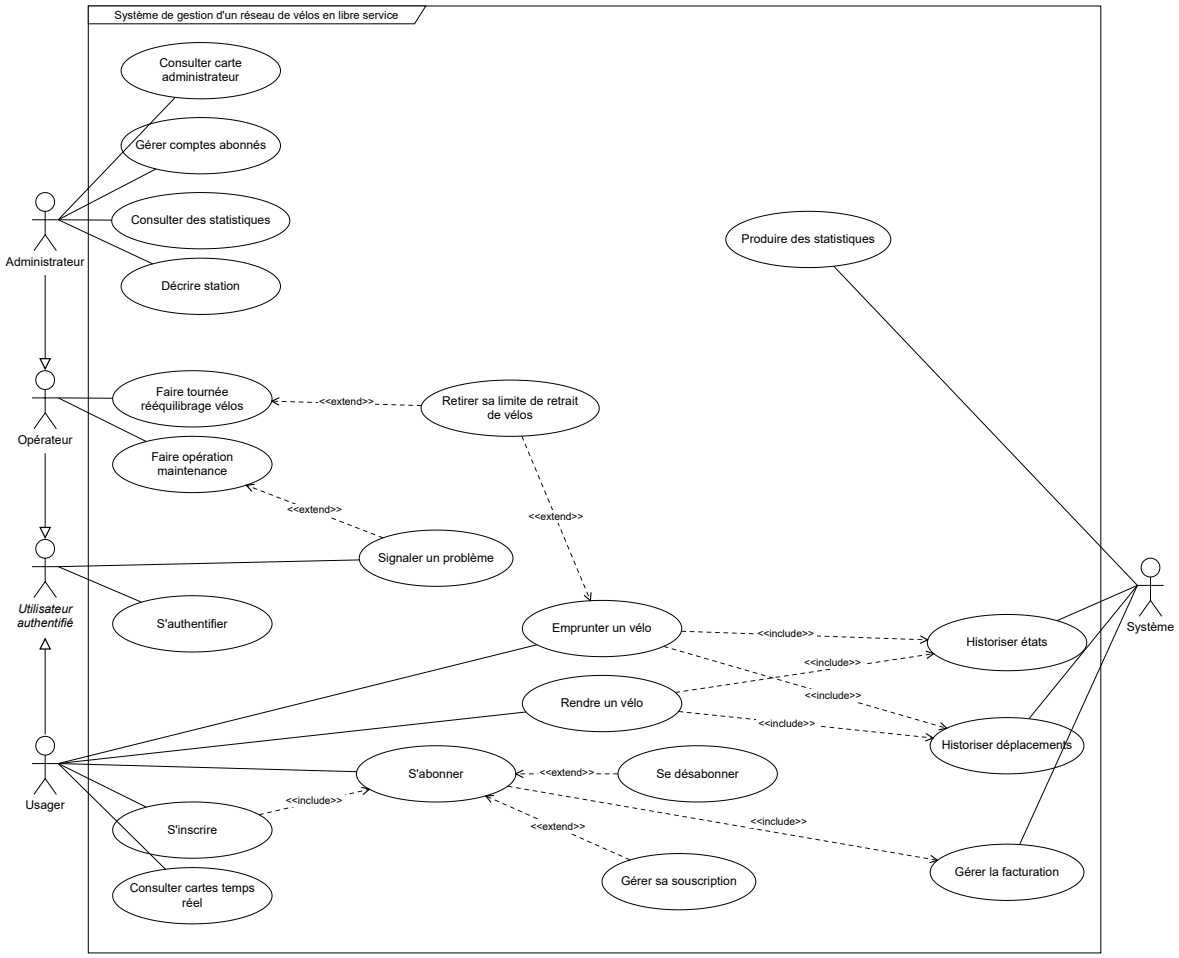


FIGURE 1 – Diagramme de cas d'utilisation

À rédiger : commenter le diagramme : regroupement des cas par acteur, relations « include » / « extend » éventuelles.

2.2 Règles métier

Les règles suivantes, issues du sujet, sont garanties par la base elle-même et non par l'application (partie 4) :

1. un vélo ne peut être qu'à un seul endroit à la fois (sur une borne, en location ou en atelier) ;
2. une borne n'accueille qu'un seul vélo ;
3. un usager n'emprunte qu'un vélo à la fois, et seulement avec un abonnement valide ;
4. un vélo signalé défectueux ou en atelier ne peut pas être emprunté ;
5. la durée et le coût d'une location sont calculés automatiquement à la restitution ;
6. tout changement d'état ou de position d'un vélo est historisé.

3 Conception de la base de données

3.1 Modèle conceptuel

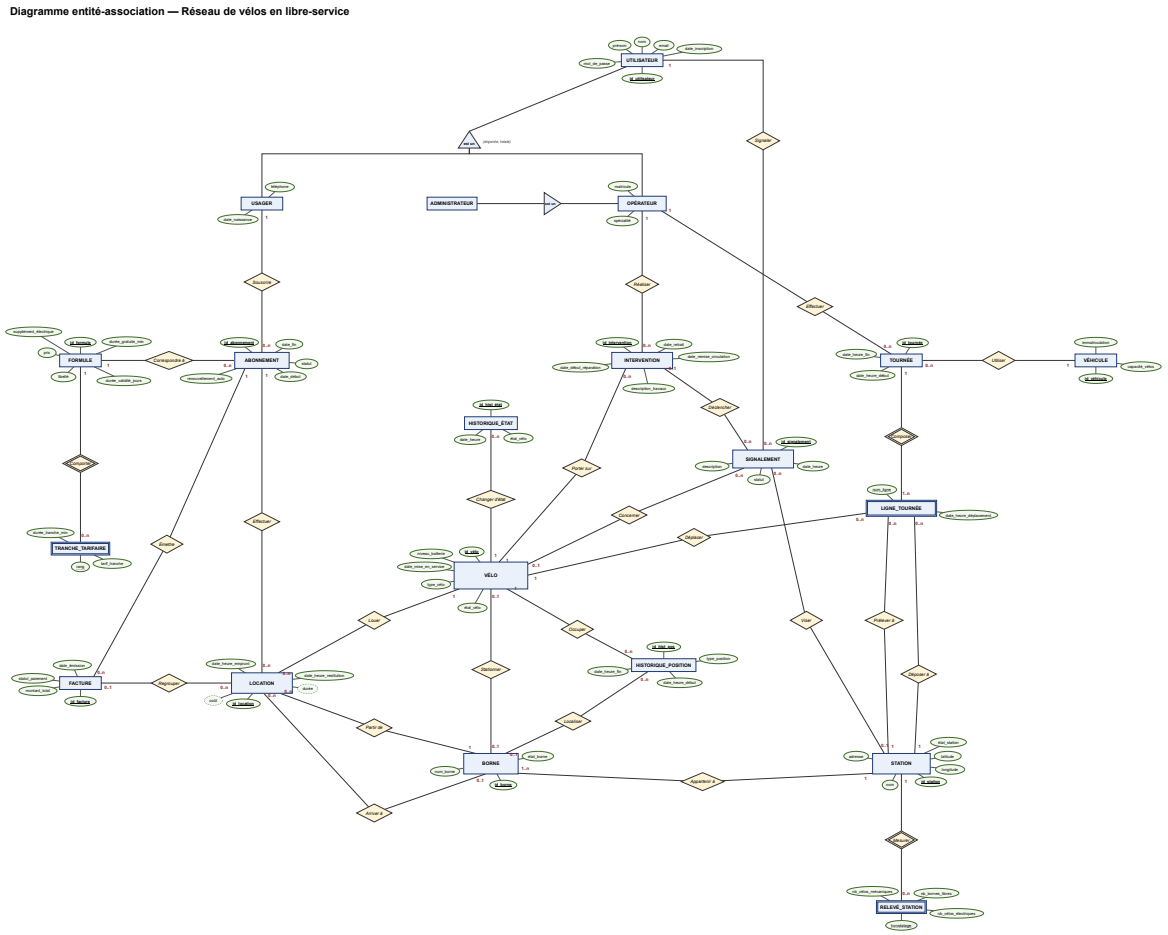


FIGURE 2 – Schéma entité-association

À rédiger : présenter les entités principales et les associations porteuses de sens (location entre abonnement, vélo et bornes ; historiques ; tournée et lignes de tournée) ; justifier les cardinalités clés et la spécialisation des utilisateurs en usager / opérateur / administrateur.

3.2 Modèle logique

MLD — Système de gestion d'un réseau de vélos en libre-service

Schéma relationnel (cible MySQL/MariaDB) · souligné = clé primaire · # = clé étrangère (flèche vers la table référencée) · [U] unique · [NULL] facultatif · [calculé] alimenté par trigger

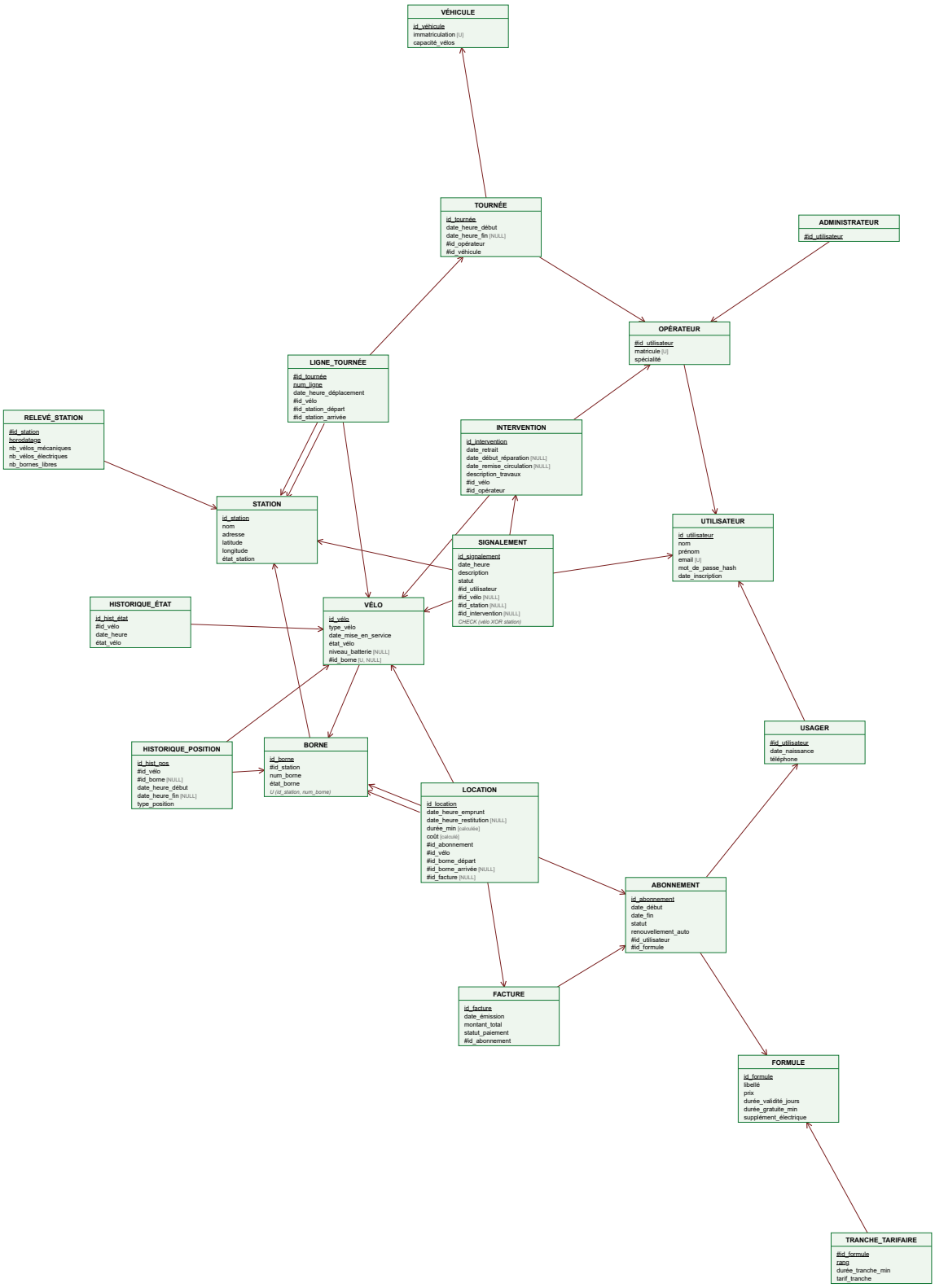


Schéma relationnel — notation textuelle

UTILISATEUR (id_utilisateur, nom, prénom, email, mot_de_passe_hash, date_inscription)
USAGER (#id_utilisateur, date_naissance, téléphone)
OPÉRATEUR (#id_utilisateur, matricule, spécialité)
ADMINISTRATEUR (#id_utilisateur)
STATION (id_station, nom, adresse, latitude, longitude, état_station)
BORNE (id_borne, #id_station, num_borne, état_borne)
RELEVÉ_STATION (#id_station, horodatage, nb_vélos_mécaniques, nb_vélos_électriques, nb_bornes_libres)
VÉLO (id_vélo, type_vélo, date_mise_en_service, état_vélo, niveau_batterie, #id_borne)
HISTORIQUE_ÉTAT (id_hist_état, #id_vélo, date_heure, état_vélo)
HISTORIQUE_POSITION (id_hist_pos, #id_vélo, #id_borne, date_heure_début, date_heure_fin, type_position)
FORMULE (id_formule, libellé, prix, durée_validité_jours, durée_gratuite_min, supplément_électrique)
TRANCHE_TARIFAIRE (#id_formule, rang, durée_tranche_min, tarif_tranche)
ABONNEMENT (id_abonnement, date_début, date_fin, statut, renouvellement_auto, #id_utilisateur, #id_formule)
LOCATION (id_location, date_heure_emprunt, date_heure_restitution, durée_min, coût, #id_abonnement, #id_vélo, #id_borne_départ, #id_borne_arrivée, #id_facture)
FACTURE (id_facture, date_émission, montant_total, statut_paiement, #id_abonnement)
SIGNALEMENT (id_signalement, date_heure, description, statut, #id_utilisateur, #id_vélo, #id_station, #id_intervention)
INTERVENTION (id_intervention, date_retrait, date_début_réparation, date_remise_circulation, description_travaux, #id_vélo, #id_opérateur)
TOURNÉE (id_tournée, date_heure_début, date_heure_fin, #id_opérateur, #id_véhicule)
VÉHICULE (id_véhicule, immatriculation, capacité_vélos)
LIGNE_TOURNÉE (#id_tournée, num_ligne, date_heure_déplacement, #id_vélo, #id_station_départ, #id_station_arrivée)

Choix de passage MCD → MLD :

- Héritage par tables filles : USAGER, OPÉRATEUR référencent UTILISATEUR ; ADMINISTRATEUR référence OPÉRATEUR (un administrateur est un opérateur).
- BORNE reçoit un identifiant technique (id_borne) avec unicité (id_station, num_borne) ; les entités faibles sans dépendants (RELEVÉ_STATION, TRANCHE_TARIFAIRE, LIGNE_TOURNÉE) gardent une clé composite.

À rédiger : expliquer le passage au relationnel (héritage par tables filles partageant la clé `id_utilisateur`, tables d'association, clés composées comme `tranche_tarifaire(id_formule, rang)` et `releve_station(id_station, horodatage)`).

3.3 Normalisation

Le schéma respecte la troisième forme normale : chaque table décrit une seule entité ou association, tout attribut non clé dépend de toute la clé et de rien d'autre que la clé. Quelques attributs dérivables ont été conservés volontairement ; ils sont calculés par la base, jamais saisis, ce qui écarte les anomalies de mise à jour que la normalisation vise à éviter :

- `location.duree_min` est une **colonne générée** (`GENERATED ALWAYS AS ... STORED`) à partir des dates d'emprunt et de restitution : elle ne peut pas diverger de ces dates, et elle est indexable pour les statistiques ;
- `location.cout` est calculé par déclencheur à la restitution et **figé** : il dépend de la tarification *au moment du trajet*, qui peut évoluer ensuite ; le recalculer fausserait les factures passées ;
- `location.velo_en_cours` et `abo_en_cours` sont des colonnes générées, non nulles seulement pendant une location : un index `UNIQUE` sur chacune garantit « au plus une location en cours » par vélo et par abonnement (MariaDB n'offre pas d'index unique partiel) ;
- `abonnement.statut` : la résiliation n'est pas déductible des dates, et l'état « expiré » est matérialisé par un événement planifié pour permettre un accès indexé aux abonnements actifs.

3.4 Contraintes d'intégrité

À rédiger : détailler les contraintes : clés étrangères et leurs actions (`ON DELETE CASCADE` pour les bornes et relevés d'une station), `CHECK` (coordonnées, batterie réservée aux vélos électriques, position cohérente avec l'état), unicités (`uq_velo_borne` : une borne, un vélo).

3.5 Index

À rédiger : présenter la stratégie d'indexation : index couvrant chaque clé étrangère, index orientés requêtes (recherche de l'abonnement valide, file des signalements, historiques par vélo et par date). Les index couvrants ajoutés pour les statistiques sont étudiés en partie 8.

4 Implémentation SQL

Les règles métier sont implémentées dans la base : l'application n'exécute aucune écriture directe sur les tables sensibles et appelle des procédures stockées, dont elle affiche tels quels les messages d'erreur levés par `SIGNAL`.

4.1 Déclencheurs et historisation

- `trg_velo_ai`, `trg_velo_au` : historisation de chaque changement d'état (`historique_etat`) et de position (`historique_position`) d'un vélo ; chaque changement de position ferme

la ligne ouverte (date de fin renseignée) et en ouvre une nouvelle ;
- `trg_location_bi`, `trg_location_bu` : contrôle des règles d'emprunt (abonnement valide, un seul vélo à la fois, vélo disponible) et calcul automatique du coût à la restitution ;
- `trg_abonnement_bi` : un seul abonnement actif à la fois par usager ;
- `trg_signalement_ai` : un vélo signalé défectueux ne peut plus être emprunté.

Pendant le chargement massif des données factices, les procédures de génération positionnent la variable de session `@seed_mode` : les déclencheurs sont alors inactifs pour cette session uniquement, le générateur produisant lui-même des historiques cohérents. Cela évite plusieurs millions d'exécutions de déclencheurs.

À rédiger : extrait commenté de `trg_velo_au`.

4.2 Procédures et fonctions métier

- locations : `sp_emprunter_velo`, `sp_rendre_velo` (transactions, verrouillage du vélo et de la borne) ;
- abonnements : `sp_souscrire_abonnement`, `sp_renouveler_abonnement`, `sp_resilier_abonnement`, `sp_expirer_abonnements` ;
- facturation : `fn_cout_formule`, `fn_calculer_cout` (durée gratuite, tranches, supplément électrique), `sp_facturer_locations` ;
- maintenance : `sp_signaler_probleme`, `sp_ouvrir_intervention`, `sp_demarrer_reparation`, `sp_cloturer_intervention` ;
- régulation : `sp_stations_a_reequilibrer`, `sp_proposer_deplacements`, `sp_demarrer_tournee`, `sp_deplacer_velo_tournee`, `sp_terminer_tournee` ;
- statistiques : six procédures `sp_stat_*` (partie 8).

Le coût d'un trajet suit la formule de l'abonnement : les `duree_gratuite_min` premières minutes sont gratuites ; au-delà, les tranches s'appliquent dans l'ordre de leur rang, la dernière se répétant tant que le trajet continue (toute tranche commencée est due) ; le supplément électrique s'ajoute pour un vélo électrique. Par exemple, avec 30 minutes gratuites et deux tranches de 30 minutes à 1 € puis 2 €, un trajet de 100 minutes coûte $1 + 2 \times \lceil 40/30 \rceil = 5$ €.

À rédiger : présenter en détail l'emprunt et la restitution (déroulé de la transaction, verrous, erreurs levées).

4.3 Événements planifiés

Trois événements automatisent les traitements périodiques : `ev_expirer_abonnements`, `ev_facturer_locatio` et `ev_releve_stations` (relevé de l'occupation des stations, qui alimente la statistique de remplissage). Ils nécessitent l'activation de l'ordonnanceur (`event_scheduler=ON` dans la configuration du serveur).

4.4 Sécurité

- l'application se connecte avec un compte dédié, `velo_app`, qui n'a que `SELECT` et `EXECUTE` sur la base, plus des droits d'écriture ciblés : insertion, modification et suppression sur `station` et `borne` ; modification de certaines *colonnes* seulement de `utilisateur`, `usager` et `signalement` ;

— mots de passe hachés avec bcrypt, régénération de la session à la connexion, limitation des tentatives ;
— jeton CSRF sur chaque formulaire, requêtes préparées uniquement, échappement automatique des gabarits, en-têtes HTTP de sécurité.

5 Jeu de données factices

Les données sont générées dans la base par les procédures `sp_seed_*` (`seed.sql`) : référentiel (formules, tranches), stations et bornes, flotte, utilisateurs, abonnements, locations, historiques, facturation, maintenance, régulation et relevés. Plusieurs tailles sont proposées (`sp_seed_petit`, `sp_seed_demo`, `sp_seed_gros`), et `sp_seed_verifier` contrôle la cohérence du résultat.

Pour la génération massive, l'insertion `INSERT ... SELECT` en isolation `REPEATABLE READ` posait un verrou partagé sur chaque ligne lue et faisait déborder la table des verrous d'InnoDB (erreur 1206) : les procédures concernées passent en `READ COMMITTED`, où la lecture reste cohérente sans verrou.

Le tableau 2 donne les volumes de la base utilisée pour les mesures de performances ; il est lu dans `data/contexte.csv`, produit par le script de mesure.

Mesure	Valeur
Date de la mesure	"2026-11-07 12 :29"
Fin des périodes mesurées	2026-11-07
Répétitions par mesure	5
Serveur	10.4.32-MariaDB
PHP	8.5.0
Buffer pool InnoDB (Mo)	4096
Taille de la base (Mo)	4779
Locations	4993545
Historique des états	9994149
Historique des positions	9993695
Factures	4167903
Abonnements	2621808
Relevés de stations	2595600
Vélos	6000
Stations	300

TABLE 2 – Contexte et volumes de la base mesurée

À rédiger : décrire comment les données sont rendues réalistes (répartition des trajets dans la journée, heures de pointe, durées, part de vélos électriques, pannes).

6 Application web

6.1 Architecture

L'application suit une architecture en couches autour du micro-framework Slim 4 :

— `public/index.php` : point d'entrée unique ;
— `config/` : paramètres (lus depuis `.env`), conteneur d'injection de dépendances, chaîne de middlewares (erreurs, en-têtes de sécurité, session, CSRF) et routes groupées par rôle ;
— `src/Controller/` : un contrôleur par domaine (stations, locations, abonnements, statistiques…) ;
— `src/Repository/` : toutes les requêtes SQL et les appels de procédures, ce qui isole l'accès aux données ;
— `templates/` : gabarits Twig (mise en page Tailwind, composants réutilisables).

À rédiger : schéma du cheminement d'une requête (route, middleware de rôle, contrôleur, repository, procédure stockée, gabarit).

6.2 Fonctionnalités par rôle

À rédiger : une capture d'écran commentée par fonctionnalité exigée : inscription et connexion ; stations en temps réel et gestion par l'administrateur (ajout, modification, suppression, recherche, filtres) ; emprunt et restitution avec coût estimé, historique des trajets ; page de statistiques (onglets, graphiques, export CSV). Ajouter les fonctionnalités supplémentaires : abonnements et factures, signalements, flotte, tableaux de bord par rôle.

6.3 Page de statistiques

La page présente les six statistiques demandées, réparties en trois onglets (« Trajets », « Stations et régulation », « Recettes et flotte ») ; seules les procédures de l'onglet affiché sont exécutées. Les résultats sont mis en cache côté application (une heure pour une période passée, cinq minutes si elle inclut le jour courant, trente secondes pour le temps réel), et un lien permet de les recalculer pour mesurer le temps réel des procédures. Chaque rapport est exportable en CSV.

7 Évolution du projet

7.1 Chronologie

Le tableau 3 retrace les étapes du projet d'après l'historique du dépôt Git.

TABLE 3 – Principales étapes du projet

Date	Étape
29/10/2026	Prototype d'interface en PHP natif : pages de base, navigation, thème sombre.
31/10/2026	Refonte de l'architecture du dépôt.
01/11/2026	Diagrammes (cas d'utilisation, modèles conceptuel et logique) ; schéma SQL, contraintes et index ; déclencheurs et procédures ; premières procédures de génération de données.
05/11/2026	Correction du chargement massif (débordement de la table des verrous InnoDB) ; compte applicatif à droits minimaux et comptes de démonstration.

Date	Étape
05/11/2026	Réécriture de l'application avec Slim 4 et Twig : authentification et rôles, stations, abonnements, locations, statistiques, signalements, flotte, tableaux de bord et gestion des comptes.
05/11/2026	Schéma entité-association ; première optimisation de la page de statistiques : onglets, pagination, cache applicatif, index couvrants, réécriture des procédures de recettes et d'indisponibilité.
06/11/2026	Tests sur la base complète (5 Go) : migration des index, configuration du serveur, réécriture des statistiques origine-destination, d'indisponibilité de la flotte et de remplissage ; banc de mesure automatisé.

À rédiger : compléter avec la contribution de chaque membre et les étapes non visibles dans l'historique (choix de conception, réunions).

7.2 D'un prototype PHP natif à une application structurée

Le premier prototype mêlait requêtes SQL et affichage dans des pages PHP. Pour la version finale, l'application a été restructurée autour de Slim 4 : routes groupées par rôle, middlewares de session, de CSRF et de contrôle d'accès, contrôleurs par domaine et repositories qui regroupent toutes les requêtes. Ce découpage rend chaque requête identifiable et mesurable, ce qui a facilité l'analyse des performances.

7.3 Problèmes rencontrés et solutions

Débordement des verrous lors du chargement massif. En isolation `REPEATABLE READ`, un `INSERT ... SELECT` pose un verrou partagé sur chaque ligne lue ; sur plusieurs millions de locations, la table des verrous d'InnoDB débordait (erreur 1206). Les procédures de génération passent en `READ COMMITTED`.

Différences entre MySQL et MariaDB. MariaDB signale une violation de contrainte `CHECK` par le code 4025 et non 3819 comme MySQL 8 : la couche d'accès aux données traduit les deux en message métier.

Droits du compte applicatif. Après une recréation de la base, le compte `velo_app` avait perdu son droit `SELECT, EXECUTE` sur la base : toutes les pages échouaient. Le script `bin/check-db.php` diagnostique désormais la connexion, les droits et la présence des données.

Page de statistiques bloquée sur la base complète. Avec le volume final, l'onglet affiché par défaut ne répondait plus. L'analyse, les corrections et les mesures font l'objet de la partie 8.

Mise à jour d'une base de plusieurs gigaoctets. Recréer la base pour modifier un index n'était plus envisageable. Les changements de schéma passent par des migrations réexécutables (`database/migrations/`) qui ne reconstruisent un index que si sa définition diffère, sans bloquer la table (`ALGORITHM=INPLACE, LOCK=NONE`).

8 Analyse des performances

Cette partie constitue le rapport d'analyse des performances demandé par le sujet. Elle porte sur les requêtes les plus coûteuses du système, les six statistiques, exécutées sur la base complète, et retrace les mesures qui ont conduit à chaque correction.

8.1 Protocole de mesure

- **Base** : jeu de données complet (tableau 2), environ 5 Go.
- **Serveur** : MariaDB 10.4.32 (XAMPP) sous Windows 11, 32 Go de RAM, cache InnoDB de 4 Go, ordonnanceur d'événements actif (`event_scheduler=ON`) : un relevé des stations, planifié toutes les 15 minutes, peut ponctuellement s'exécuter pendant une mesure, ce que la médiane absorbe.
- **Procédures** : version finale de la branche `dev`, rechargée avant la mesure. Un premier passage avait été lancé par erreur sur les anciennes procédures : il redonnait les temps d'avant correction (415 s pour l'indisponibilité sur 30 jours), ce qui confirme au passage l'effet des réécritures.
- **Méthode** : chaque procédure est appelée directement, sans le cache de l'application, par le script `bin/bench-stats.php`. Pour chaque longueur de période (7, 30, 90 et 365 jours), le premier appel est noté à part, car il inclut le chargement des pages en mémoire ; la valeur retenue est la **médiane** des appels suivants.
- **Exécution de l'application** : les temps de page sont mesurés avec `curl` sur le serveur PHP intégré, connecté en administrateur, avec et sans le cache applicatif.
- **Vérification** : chaque requête réécrite a été comparée ligne à ligne à l'ancienne version sur plusieurs périodes ; les résultats sont identiques.

La page de statistiques s'ouvre par défaut sur les 30 derniers jours : c'est la période de référence. L'objectif fixé est un affichage de chaque onglet en moins de 5 secondes sans cache.

8.2 Constat initial

Sur la base complète, l'onglet « Trajets », affiché par défaut, ne répondait plus : la requête HTTP restait en attente plus de deux minutes. La mesure de chaque procédure isolément a révélé trois requêtes problématiques : les trajets origine-destination (plus de 60 s), l'indisponibilité de la flotte (plus de 100 s) et, dans une moindre mesure, les recettes par formule (8,8 s). Deux causes, de natures différentes, se cumulaient.

8.3 Configuration du serveur

Le paramètre `innodb_buffer_pool_size`, mémoire qu'InnoDB consacre au cache des données et des index, valait 16 Mo, valeur par défaut de XAMPP, pour une base de près de 5 Go : presque chaque page lue l'était sur le disque. Il a été porté à 4 Go (et `innodb_log_file_size` à 512 Mo, environ un quart du cache). La figure 4 compare les procédures dont la version n'a pas changé entre les deux mesures.

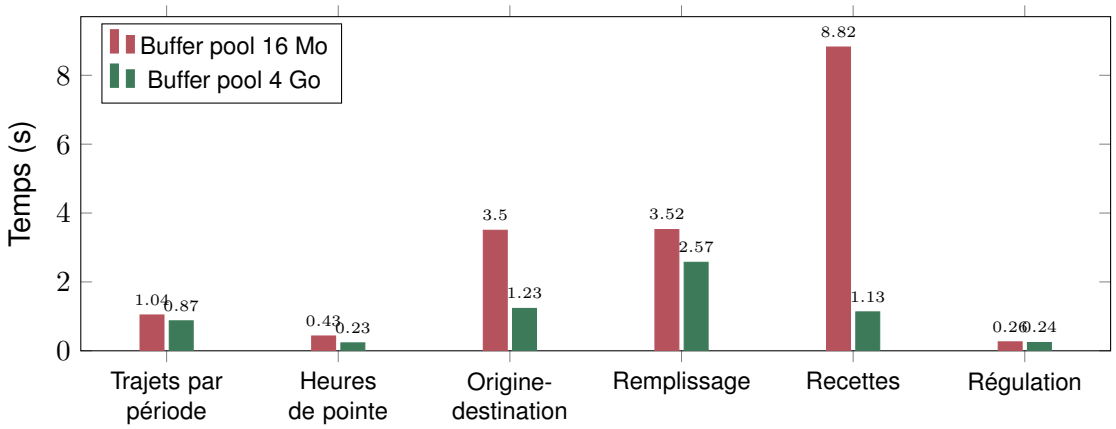


FIGURE 4 – Effet du cache InnoDB (période de 30 jours, mêmes versions des procédures)

Le gain est le plus fort pour les requêtes qui relisent beaucoup de lignes par clé primaire, comme les recettes (8,8 s $\rightarrow$ 1,1 s) : chaque facture de la période est relue dans la table, à des emplacements dispersés. Ce réglage ne suffisait toutefois pas : avec toutes les données en mémoire, l'indisponibilité de la flotte prenait encore 22 s. Le problème restant tenait à la forme des requêtes.

8.4 Réécriture des requêtes

La figure 5 résume les gains obtenus par réécriture, chaque fois à configuration identique.

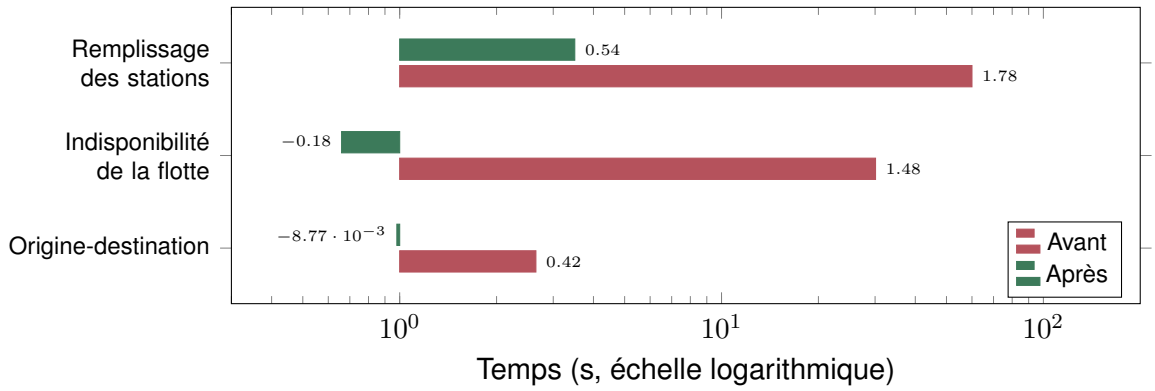


FIGURE 5 – Temps avant et après réécriture (30 jours). Origine-destination mesurée avec un cache de 16 Mo (avant : interrompue après 60 s) ; les deux autres avec 4 Go.

8.4.1 Trajets origine-destination

La requête initiale joignait `location` aux bornes et aux stations de départ et d'arrivée, puis filtrait sur la période. Son plan d'exécution (`EXPLAIN`) montre que l'optimiseur partait des stations, puis des bornes, puis des locations via l'index des arrivées :

```
table  type   key                   rows  Extra
sa     index  uq_station_nom         300  Using temporary; Using filesort
ba     ref    uq_borne_station_num    13  Using index
l      ref    idx_loc_arrivee_resti  243  Using where
```

Ce chemin lit *toutes* les locations de chaque borne (environ 5 millions) et ne filtre la date qu'ensuite, alors que la période n'en contient que 406 000. La version réécrite :

- impose l'ordre de jointure (`STRAIGHT_JOIN`) : la lecture part des locations de la période, par une plage sur l'index couvrant `idx_loc_emprunt`, sans relire la table ;
- regroupe par paire de *stations* (au plus 300^2 groupes) et non par paire de bornes (jusqu'à $7\,800^2$), ce qui garde la table temporaire en mémoire ; une première réécriture qui regroupait d'abord par bornes prenait encore 27 s ;
- ne joint les noms des stations qu'au résultat agrégé.

```
SELECT sd.id_station, sd.nom, sa.id_station, sa.nom,
       x.nb_trajets, ROUND(x.duree_moyenne_min, 1)
  FROM (SELECT STRAIGHT_JOIN bd.id_station AS id_depart, ba.id_station AS id_arrivee,
               COUNT(*) AS nb_trajets, AVG(l.duree_min) AS duree_moyenne_min
          FROM location l
          JOIN borne bd ON bd.id_borne = l.id_borne_depart
          JOIN borne ba ON ba.id_borne = l.id_borne_arrivee
         WHERE l.date_heure_emprunt >= p_debut AND l.date_heure_emprunt < p_fin
         GROUP BY bd.id_station, ba.id_station) x
  JOIN station sd ON sd.id_station = x.id_depart
  JOIN station sa ON sa.id_station = x.id_arrivee
 ORDER BY x.nb_trajets DESC, sd.nom, sa.nom
 LIMIT p_limite;
```

Le nouveau plan lit la plage de dates de l'index (`range`, `Using index`) puis accède aux bornes par clé primaire (`eq_ref`). Temps : plus de 60 s $\rightarrow$ 3,5 s avec le cache de 16 Mo, 1,2 s avec 4 Go.

8.4.2 Indisponibilité de la flotte

Le taux d'indisponibilité est la part du temps passée « en atelier » ou « défectueux », calculée à partir de `historique_etat`. La version initiale reconstituait *tous* les intervalles d'état de la période avec la fonction de fenêtre `LEAD` : sur 30 jours, 813 000 changements d'état, dont la plupart (« disponible » / « en location ») sont dus aux locations. La décomposition de la requête a montré deux coûts : la relecture de chaque ligne par clé primaire pour obtenir `id_velo` (6 s), absent de l'index utilisé, et la fonction de fenêtre elle-même (environ 4 s, même avec un index couvrant). Ajouter un index ne pouvait donc pas suffire.

Or seuls les intervalles d'indisponibilité comptent au numérateur : il n'y a que 174 passages en atelier ou défectueux sur 30 jours (788 sur un an). La procédure a été réécrite sans fonction de fenêtre :

- **numérateur** : pour chaque passage dans un état indisponible pendant la période, ainsi que pour l'état de chaque vélo à l'ouverture de la période, la fin de l'intervalle est l'événement suivant du même vélo, trouvé par une seule lecture de l'index `(id_velo, date_heure)` ;
- **dénominateur** : les intervalles d'un vélo se suivent sans interruption de son premier événement connu jusqu'à la fin de la période ; le temps observé vaut donc *fin* $-$ max(*début*, *premier événement*), soit une lecture d'index par vélo.

Un dernier écueil est apparu : dans une sous-requête corrélée, MariaDB n'optimise pas `MIN(date_heure)` en une simple lecture de la première entrée d'index, et parcourait les quelque 1 600 entrées de chaque vélo (10 millions au total, 4,9 s). L'écriture équivalente `ORDER BY date_heure LIMIT 1` ne lit qu'une entrée.

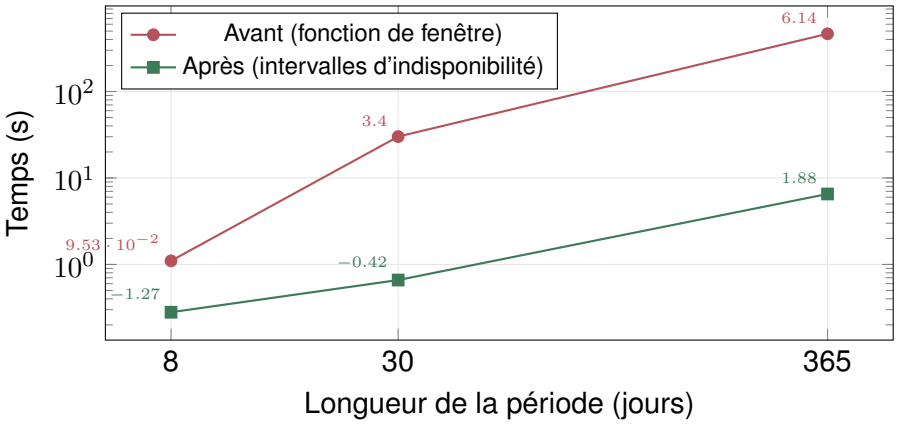


FIGURE 6 – Indisponibilité de la flotte selon la longueur de la période (cache de 4 Go)

Les résultats sont identiques à ceux de l'ancienne version sur 8 jours, 30 jours et un an ; le temps passe de 30 s à 0,7 s sur 30 jours et de 465 s à 6,5 s sur un an (figure 6).

8.4.3 Remplissage des stations

Même défaut que pour les trajets origine-destination : l'optimiseur partait des 300 stations et lisait par clé primaire *tous* les relevés de chacune (2,6 millions) avant de filtrer la date. Les relevés de la période (777 000) sont désormais agrégés d'abord, par une plage sur l'index couvrant `idx_releve_horodatage`, puis joints aux stations : 2,65 s → 0,98 s, sur les mêmes 7 200 lignes de résultat.

8.4.4 Index couvrants

Deux index ont été étendus pour que les statistiques ne lisent que l'index, sans relire la table :

- `location.idx_loc_emprunt` : `(date_heure_emprunt, date_heure_restitution, id_borne_depart, id_borne_arrivee, duree_min, cout)` ;
- `releve_station.idx_releve_horodatage` : `(horodatage, id_station, nb_velos_mecaniques, nb_velos_electriques, nb_bornes_libres)`.

La base existante a été mise à jour par une migration ré-exécutable, sans la recréer (partie 7).

8.5 Résultats

8.5.1 Temps des requêtes principales

Le tableau 4 donne le temps de chaque procédure sur la période de référence, après toutes les corrections.

Procédure	1er appel (s)	Médiane (s)	Min (s)	Max (s)	Lignes
Trajets par période	0.85	0.86	0.79	1.22	27
Heures de pointe	0.31	0.26	0.24	0.35	19
Trajets origine-destination	2.13	1.96	1.87	2.32	500
Remplissage des stations	1.14	1.07	0.97	1.17	7,200
Recettes par formule	1.35	1.55	1.46	1.59	4
Indisponibilité de la flotte	0.54	0.59	0.56	0.61	3
Stations à rééquilibrer	0.07	0.05	0.05	0.07	113
Déplacements proposés	0.11	0.14	0.11	0.18	0

TABLE 4 – Temps d'exécution des procédures statistiques sur 30 jours

Les écarts avec les temps cités plus haut (origine-destination : 1,2 s lors de la mise au point, 2,0 s ici) tiennent à la variabilité d'une machine de bureau ; les ordres de grandeur sont stables.

Chaque onglet de la page n'exécute que ses propres procédures, plus les indicateurs clés (trajets par période et heures de pointe) affichés en tête de chaque onglet. En additionnant les médianes, on obtient le temps de calcul de chaque onglet (tableau 5).

Onglet	Procédures appelées	30 j	90 j	365 j
Trajets	indicateurs + origine-destination	3,1 s	9,7 s	30,6 s
Stations	indicateurs + remplissage + rééquilibrage + déplacements	2,4 s	7,4 s	17,5 s
Recettes	indicateurs + recettes + indisponibilité	3,3 s	8,8 s	28,0 s

TABLE 5 – Temps de calcul de chaque onglet, sans cache (somme des médianes)

Sur la période de référence de 30 jours, les trois onglets respectent l'objectif de 5 s, avec une marge de 35 à 50 %. Au-delà de trois mois, l'objectif n'est plus tenu sans cache (voir les limites ci-dessous).

8.5.2 Montée en charge

La figure 7 montre l'évolution du temps de chaque statistique avec la longueur de la période.

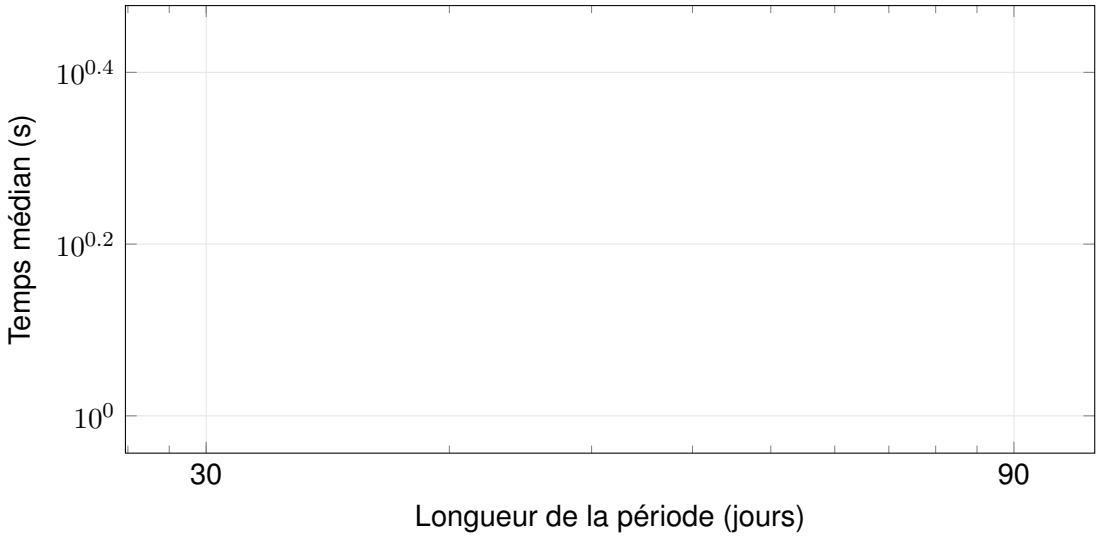


FIGURE 7 – Temps médian de chaque statistique selon la longueur de la période

Le temps croît à peu près linéairement avec le nombre de lignes de la période. Les procédures lisent désormais uniquement les lignes utiles, par des plages d'index : leur coût suit donc la taille de la période, et non plus celle de la table entière.

8.5.3 Temps d'affichage et cache applicatif

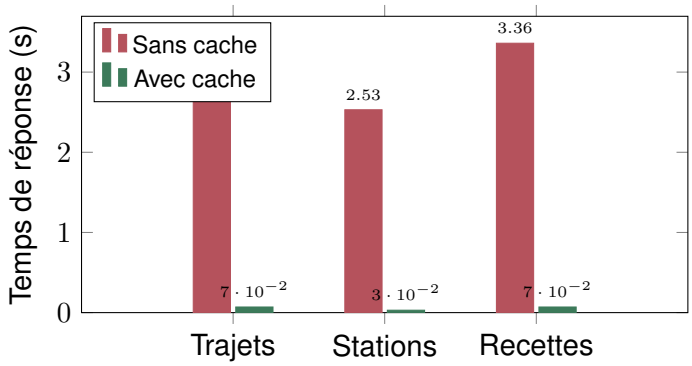


FIGURE 8 – Temps de réponse de chaque onglet de la page de statistiques (30 jours, médiane de 3 requêtes)

Mesurés de bout en bout par HTTP, avec le lien « Recalculer sans le cache », les onglets répondent en 3,0 s (Trajets), 2,5 s (Stations) et 3,4 s (Recettes) : le rendu PHP et Twig n'ajoute que quelques dixièmes de seconde au calcul des procédures. Une fois les résultats en cache, chaque onglet s'affiche en moins de 0,1 s, quelle que soit la période (figure 8). Lors d'une mesure intermédiaire, avec la requête origine-destination déjà réécrite mais pas encore le remplissage, l'onglet Stations prenait 5,1 s : c'est ce dépassement qui a motivé la dernière réécriture.

8.6 Limites et pistes d'amélioration

— Sur une période d'un an, les statistiques de trajets agrègent l'essentiel des 5 millions de locations : sans cache, les onglets prennent alors de 17 à 31 s (tableau 5), la requête origine-destination à elle seule 16,5 s. Le cache d'une heure sur les périodes passées en atténue l'effet, mais le premier calcul reste long.
— La piste naturelle est une **table d'agrégats journaliers** (trajets par jour et par paire de stations, recettes par jour et par formule), alimentée chaque nuit par un événement planifié : les statistiques liraient quelques milliers de lignes au lieu de plusieurs millions, au prix d'une dénormalisation contrôlée.
— Le partitionnement des grandes tables par date (`location`, `historique_etat`) limiterait aussi les lectures aux partitions de la période.
— Les performances dépendent fortement de la configuration du serveur : avec le cache InnoDB par défaut de 16 Mo, aucune réécriture ne suffit à tenir l'objectif.

9 Conclusion

Le système réalisé couvre l'ensemble du cahier des charges : la base de données modélise toutes les fonctionnalités demandées et en garantit les règles métier par des contraintes, des déclencheurs et des procédures stockées, et l'application web dépasse le minimum exigé en exposant, en plus de l'authentification, des stations, des locations et des statistiques, la gestion des abonnements, des signalements, de la flotte et des comptes.

Le passage à un jeu de données réaliste de plusieurs gigaoctets a été l'étape la plus formatrice : il a révélé des requêtes correctes mais inexploitables à ce volume, et imposé une démarche rigoureuse de mesure, de lecture des plans d'exécution et de réécriture, en vérifiant à chaque fois que les résultats restaient identiques.

À rédiger : bilan personnel de l'équipe, limites (statistiques sur de longues périodes, maintenance et régulation partiellement exposées dans l'interface) et perspectives (tables d'agrégats, carte des stations, application mobile…).

A Installation

Les scripts SQL s'exécutent dans cet ordre avec le compte `root` : `schema.sql`, `triggers.sql`, `procedures.sql`, `seed.sql` (puis `CALL sp_seed_petit();`), `app_user.sql` et `demo_accounts.sql`. Une base existante se met à jour par les scripts de `database/migrations/`, suivis de `procedures.sql`.

Configuration recommandée du serveur (`my.ini`, section `[mysqld]`) :

```
event_scheduler=ON              -- événements planifiés
innodb_buffer_pool_size=4G      -- XAMPP fournit 16M
innodb_log_file_size=512M       -- environ 25 % du buffer pool
```

L'application s'installe avec `composer install`, se configure dans `.env` (mot de passe du compte `velo_app`) et se lance avec `composer start`. La commande `composer check-db` vérifie la connexion, les droits et la présence des données.

B Reproduire les mesures

```
php bin/bench-stats.php --repetitions=5 --periodes=7,30,90,365
```

Le script écrit `var/bench/procedures.csv` et `var/bench/contexte.csv`. Copiés dans `docs/report/data/`, ils mettent à jour automatiquement les tableaux 2 et 4 ainsi que la figure 7 à la compilation suivante.